

Chương 6.
CÁC HỆ THỐNG QUẢN LÝ PHIÊN BẢN

Mục tiêu

□ Về kiến thức:

- Hiểu được **tầm quan trọng** và **lợi ích cốt lõi** của hệ thống quản lý phiên bản (VCS).
- Hiểu các **khái niệm cơ bản** của Subversion (SVN), Git và GitHub.
- **Phân biệt** được sự khác biệt chính giữa SVN và Git về kiến trúc và quy trình làm việc.
- Biết cách **lựa chọn** hệ thống quản lý phiên bản phù hợp với nhu cầu dự án.

Mục tiêu (tt)

□ Về kỹ năng:

- **Sử dụng thành thạo** các lệnh cơ bản của SVN và Git để quản lý phiên bản.
- **Thực hiện** quy trình làm việc nhóm cơ bản với Git và GitHub (tạo repository, commit, push, pull, pull request).
- **Giải quyết** các xung đột (conflicts) cơ bản trong cả SVN và Git.
- **Tạo và quản lý** repository trên GitHub.

Mục tiêu (tt)

❑ **Tự chủ và trách nhiệm:**

- **Chủ động** tìm hiểu và áp dụng các công cụ quản lý phiên bản vào dự án cá nhân và nhóm.
- **Có ý thức** về việc quản lý lịch sử thay đổi của mã nguồn một cách có hệ thống và trách nhiệm.
- **Tuân thủ** các quy trình làm việc nhóm đã thống nhất khi sử dụng VCS.
- **Chịu trách nhiệm** về các commit và pull request của bản thân.

Nội dung giảng dạy

1. Giới thiệu Hệ thống quản lý phiên bản
2. Các loại VCS phổ biến
3. Subversion (SVN)
4. Các khái niệm SVN
5. Quy trình làm việc SVN
6. Các lệnh SVN thường dùng
7. Git
8. Các khái niệm Git
9. Quy trình làm việc Git
10. Các lệnh Git thường dùng

Nội dung giảng dạy (tt)

11. So sánh SVN và Git
12. GitHub
13. Tính năng chính của GitHub
14. Làm việc với Git và GitHub
15. Lựa chọn VCS phù hợp
16. Thực hành SVN
17. Thực hành Git
18. Thực hành GitHub

6.1 Giới thiệu Hệ thống quản lý phiên bản

- ❑ Hệ thống quản lý phiên bản (VCS – Version Control System) là công cụ giúp theo dõi, quản lý và duy trì lịch sử thay đổi của mã nguồn và tài liệu trong quá trình phát triển phần mềm.
- ❑ **Lợi ích của VCS:**
 - **Theo dõi lịch sử thay đổi:** Lưu giữ mọi thay đổi của mã nguồn và tập tin theo thời gian.
 - **Quay lại phiên bản cũ:** Dễ dàng khôi phục các phiên bản trước khi có lỗi hoặc thay đổi không mong muốn.
 - **Hợp tác hiệu quả:** Cho phép nhiều người cùng làm việc trên dự án mà không gây xung đột.
 - **Quản lý nhánh:** Hỗ trợ tạo và quản lý các nhánh phát triển song song.
 - **Kiểm soát thay đổi:** Biết được ai, khi nào và tại sao có những thay đổi.
 - **Sao lưu và bảo vệ:** Đảm bảo an toàn cho mã nguồn và lịch sử phát triển.

6.2 Các loại VCS phổ biến

❑ Hệ thống quản lý phiên bản cục bộ (Local VCS)

- Lưu trữ lịch sử trên máy tính cá nhân.
- Không phù hợp cho làm việc nhóm.

❑ Hệ thống quản lý phiên bản tập trung (Centralized VCS)

- Một máy chủ trung tâm lưu trữ tất cả các phiên bản.
- Client kết nối đến máy chủ để làm việc.

❑ Hệ thống quản lý phiên bản phân tán (Distributed VCS)

- Mỗi người dùng có một bản sao đầy đủ của lịch sử dự án.
- Làm việc offline và hợp nhất dễ dàng hơn.

6.3 Subversion (SVN)

- ❑ Hệ thống quản lý phiên bản tập trung phổ biến.
- ❑ Sử dụng mô hình client-server.
- ❑ Lịch sử các thay đổi được lưu trữ trên máy chủ trung tâm.
- ❑ Các client làm việc trên bản sao cục bộ (working copy).

6.4 Các khái niệm SVN

- ❑ **Repository:** Kho lưu trữ trung tâm chứa tất cả các phiên bản.
- ❑ **Client:** Phần mềm tương tác với repository.
- ❑ **Working Copy:** Bản sao cục bộ của dự án.
- ❑ **Commit:** Lưu thay đổi từ working copy lên repository (tăng số phiên bản).
- ❑ **Update:** Tải thay đổi mới nhất từ repository về working copy.
- ❑ **Checkout:** Tải bản sao ban đầu của repository.

6.5 Quy trình làm việc SVN

1. **Checkout:** Lấy bản sao dự án từ repository.
2. **Edit:** Thực hiện các thay đổi trên working copy.
3. **Update:** Kiểm tra và cập nhật các thay đổi từ repository.
4. **Commit:** Gửi các thay đổi lên repository.
5. **Resolve Conflicts:** Giải quyết các thay đổi mâu thuẫn (nếu có xung đột).

6.6 Các lệnh SVN thường dùng

- ❑ `svn checkout <repository_url> <local_directory>`: Tải dự án.
- ❑ `svn update`: Cập nhật working copy.
- ❑ `svn add <file>`: Thêm tập tin mới vào quản lý.
- ❑ `svn commit -m "message"`: Lưu thay đổi với thông điệp.
- ❑ `svn status`: Xem trạng thái tập tin.

6.6 Các lệnh SVN thường dùng (tt)

- ❑ `svn diff`: Xem sự khác biệt.
- ❑ `svn log`: Xem lịch sử commit.
- ❑ `svn revert <file>`: Hoàn tác thay đổi cục bộ.
- ❑ `svn branch <branch_name>`: Tạo nhánh.
- ❑ `svn switch <branch_url>`: Chuyển nhánh.
- ❑ `svn merge <branch_url>`: Hợp nhất nhánh.

6.7 Git

- ❑ Hệ thống quản lý phiên bản phân tán phổ biến nhất hiện nay.
- ❑ Mỗi người dùng có một bản sao đầy đủ của repository và lịch sử.
- ❑ Nhánh và hợp nhất rất nhẹ nhàng và hiệu quả.
- ❑ Hỗ trợ làm việc offline tốt.

6.8 Các khái niệm Git

- ❑ **Repository:** Kho lưu trữ (cục bộ và từ xa).
- ❑ **Working Directory:** Thư mục làm việc hiện tại.
- ❑ **Staging Area:** Khu vực tập kết chuẩn bị cho commit.
- ❑ **Commit:** Lưu các thay đổi đã tập kết.
- ❑ **Branch:** Phiên bản riêng biệt của kho lưu trữ chính. Branch cho phép thử nghiệm tính năng mới, sửa lỗi, hay làm việc song song mà không ảnh hưởng đến bản chính.
- ❑ **Merge:** Kết hợp các thay đổi từ một nhánh.
- ❑ **Remote Repository:** Repository trên máy chủ (ví dụ: GitHub).

6.9 Quy trình làm việc Git

1. **Initialize/Clone:** Tạo repository mới hoặc sao chép từ xa.
2. **Modify:** Thực hiện thay đổi trong working directory.
3. **Stage:** Thêm thay đổi vào staging area (git add).
4. **Commit:** Lưu các thay đổi đã tập kết vào repository cục bộ (git commit).
5. **Branch/Merge:** Tạo và hợp nhất các nhánh.
6. **Push/Pull:** Đồng bộ hóa với remote repository (git push, git pull).

6.10 Các lệnh Git thường dùng

- ❑ `git init`: Khởi tạo repository mới.
- ❑ `git clone <repository_url> <local_directory>`: Sao chép repository.
- ❑ `git add <file> / git add .`: Thêm vào staging area.
- ❑ `git commit -m "message"`: Lưu các thay đổi đã thêm vào staging area.
- ❑ `git status`: Xem trạng thái.
- ❑ `git diff`: Xem khác biệt.

6.10 Các lệnh Git thường dùng (tt)

- ❑ `git log`: Xem lịch sử commit.
- ❑ `git branch`: Quản lý nhánh.
- ❑ `git checkout <branch_name>`: Chuyển nhánh.
- ❑ `git merge <branch_name>`: Hợp nhất nhánh.
- ❑ `git push <remote_name> <branch_name>`: Đẩy lên remote.
- ❑ `git pull <remote_name> <branch_name>`: Kéo từ remote.

6.11 So sánh SVN và Git

Tính năng	Subversion (SVN)	Git
Kiến trúc	Tập trung (Centralized)	Phân tán (Distributed)
Repository	Một repository trung tâm duy nhất	Mỗi người dùng có repository cục bộ đầy đủ
Nhánh (Branch)	Nặng nề, là bản sao của thư mục	Nhẹ nhàng, chỉ là con trỏ đến commit
Hiệu suất	Chậm hơn (đặc biệt offline)	Nhanh hơn (hầu hết thao tác cục bộ)
Tính linh hoạt	Ít linh hoạt	Rất linh hoạt
Offline	Hạn chế	Tốt

6.12 GitHub

- ❑ Nền tảng web hosting cho các repository
- ❑ Git. Cung cấp công cụ cộng tác mạnh mẽ.
- ❑ Mạng xã hội cho các nhà phát triển.
- ❑ Hỗ trợ quản lý dự án, theo dõi lỗi, review code.

6.13 Tính năng chính của GitHub

- ❑ **Repository Hosting:** Lưu trữ repository Git.
- ❑ **Collaboration:** Pull Requests, Issues, Code Reviews.
- ❑ **Issue Tracking:** Theo dõi lỗi và yêu cầu tính năng.
- ❑ **Pull Requests:** Đề xuất và thảo luận thay đổi.
- ❑ **Code Review:** Xem xét và nhận xét mã.
- ❑ **Forking:** Tạo bản sao cá nhân của repository.
- ❑ **Wiki:** Tài liệu dự án.
- ❑ **GitHub Pages:** Hosting trang web tĩnh.
- ❑ **GitHub Actions:** Tự động hóa quy trình (CI/CD).

6.14 Làm việc với Git và GitHub

1. Tạo repository trên GitHub.
2. `git clone` về máy cục bộ.
3. Thực hiện thay đổi cục bộ (`add`, `commit`).
4. `git push` lên GitHub.
5. Tạo Pull Request (nếu làm việc nhóm).
6. Thảo luận và Merge Pull Request.
7. `git pull` cập nhật thay đổi từ GitHub.

6.15 Lựa chọn VCS phù hợp

❑ Subversion:

- Phù hợp cho dự án nhỏ và trung bình, yêu cầu kiểm soát tập trung.
- Dễ học cho người mới bắt đầu.
- Hạn chế về khả năng làm việc offline và quản lý nhánh phức tạp.

❑ Git:

- Phù hợp cho hầu hết các loại dự án, đặc biệt là dự án lớn và phức tạp.
- Linh hoạt, hiệu suất cao, hỗ trợ làm việc offline tốt.
- Đường cong học tập ban đầu có thể dốc hơn.

6.16 Thực hành SVN

- ❑ [Click vào đây để xem nội dung thực hành](#)

6.17 Thực hành Git

- ❑ [Click vào đây để xem nội dung thực hành](#)

6.18 Thực hành GitHub

- ❑ [Click vào đây để xem nội dung thực hành](#)